Graphs and Depth First Search in C

Mo Wang

Spring 2026

Introduction

This report implements shortest-path search on weighted, bidirectional railway graphs in C, starting from a UTF-8-safe hash-indexed city map built from CSV and progressing through a series of depth-first search refinements. We begin with a timeout-limited DFS that prunes by remaining time but risks redundant exploration, thereafter introduce loop-detection and a constant-time visited flag to eliminate oscillations, and finally add branch-and-bound to prune entire sub-trees once a short and competitive path is found. Measured on the Sweden network and organized by hop count, the benchmarks show how pruning and cycle control improve practical execution times while revealing the inherent path-enumeration limits of DFS on larger, denser graphs.

Basic concepts: Graph

A graph has a set of nodes (vertices) as well as a set of edges serving as a connection between a pair of nodes. Edges can be both directed and undirected: directed edges have a directed orientation from one node to another one where traversal only can be accomplished along the direction, while undirected edges don't have direction orientation. Additionally, edges can be weighted with a non-negative value indicating a cost to reach a node, such as travel time in minutes.

Traversing the graph involves finding a path through the graph. A path is a sequence of nodes, where each consecutive pair is connected by an edge. Paths can connect nodes that are not directly connected by a single edge.

In a directed graph, there may be a connection from A to B without a corresponding connection from B to A. Edges can be chained to form paths, enabling traversal from a source to a destination even when they are not directly connected.

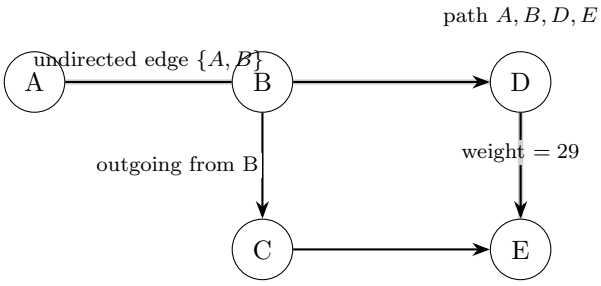

Figure 1: Graph concepts: nodes (A,B,C,D,E), an undirected edge $\{A, B\}$, directed edges ($B \to C$, $B \to D$, $C \to E$, $D \to E$), a path $A \to B \to D \to E$, and an example weight on edge $D \to E$.

Graph example: Map

A map is a trivial example of a graph. Since cities on a map are connected by a series of roads and railway networks, it can easily be represented by a graph where cities takes the function as nodes while railway connections between cities as edges.

Data Structure

A rail network map can be modeled into data structures by first representing it as a directed (bidirectional) weighted graphs graph by mapping cities as nodes or vertices, as well as railway connections as edges.

Each node, as a city (`struct city`), stores its name and an adjacency list of outgoing edges in a linked list using edge structure blocks. For efficient indexing and traversal operations, each city also stores a compact identifier `id` and a `visited` flag used during graph search algorithms. Additionally, `next_in_bucket` links cities that fall into the same hash-table bucket, which is used only for name lookup and is not part of the conceptual graph.

```
struct city {
    char  *name;            // UTF-8 city name (bytewise compare)
    conn  *adj;             // adjacency list
    int    id;              // compact index (0..city_count-1)
    bool visited;           // true if visited during search
    city  *next_in_bucket;  // next in hash chain
};
```

Edges in the graph are represented by `conn` structures stored in the adjacency lists. Each `conn` element represents a directed connection from the current city to a destination city (`dst`) and stores the associated weight `dist`, representing the travel distance in minutes. The `next` pointer links connections together to form the adjacency list.

```
struct conn {
    city *dst;
    int   dist;              // minutes (positive integer)
    conn *next;
};
```

The graph is managed by the `map_t` structure, which acts as the graph container and provides a hash-based index from city names to vertices. It stores an array of hash-table buckets (`buckets`) for fast lookup, the table size same as modulo value `mod`, and global counts of cities (`city_count`) and edges (`edge_count`). In this case, a hash table can be built to efficiently index nodes by its name.

```
typedef struct {
    city **buckets;    // hash table buckets
    int  mod;        // table size (prime recommended)
    int  city_count; // number of cities when initialization to allocate memory
    long edge_count; // number of undirected edges added when initialization to all
} map_t;
```

Lookup procedure

In the graph representation of map, each city corresponds to a *node* (vertex), and each connection to another city is stored in its *adjacency list*. To build the map, the node that represents a city must be retrieved. The lookup procedure therefore acts as the mapping relationship of name to node: given a city name, it searches the appropriate hash-table bucket for the existing node. If the node is found, it is returned so that new edges can be attached into the adjacency list. If it is not found during node creation, a new city node is allocated and inserted into the hash table. This ensures that every city name in the map corresponds to exactly one graph node, and edges in the CSV file always connect well-defined nodes.

```
// Look up city by name, optionally create if missing
city *map_lookup(map_t *m, const char *name, bool create_if_missing) {
    unsigned h = hash_str(name, (unsigned)m->mod);
    // Search in hash bucket
    for (city *cur = m->buckets[h]; cur; cur = cur->next_in_bucket) {
        if (strcmp(cur->name, name) == 0) return cur;  // Found
    }
    if (!create_if_missing) return NULL;  // Not found

    // Create new city
    city *c = (city*)calloc(1, sizeof(city));
    if (!c) return NULL;
```

```
    c->name = strdup(name);
    c->adj  = NULL;
    c->id   = -1;
    c->next_in_bucket = m->buckets[h];  // Insert at head
    m->buckets[h] = c;
    m->city_count++;
    return c;
}
```

Hashing

A hash table between city names in char arrays to their nodes in the graph is built so that insertions and lookups can be performed in amortized constant time complexity $O(1)$. The hash function processes each byte of a UTF-8 char array and accumulates with base 31. Declaring each byte as unsigned avoids interpreting each byte as a negative value when the most significant bit of the byte is 1, since char type without unsigned can reinterpret each byte as a signed 8-bit integer. The capacity or mod value is chosen to a large prime number to both keep the load factor low and reducing numbers of collisions.

The function below treats each byte as `unsigned char` (8-bit unsigned integer) and roll modulo the table capacity:

```
static unsigned hash_str(const char *name, unsigned mod) {
    unsigned h = 0;
    for (size_t i = 0; name[i] != '\0'; ++i) {
        unsigned char c = (unsigned char)name[i];
        h = (h * 31u + c) % mod;
    }
    return h % mod;
}
```

Buckets vs linear probing

The hash table can either be built using buckets or using linear probing. In this implementation, the bucket method is used because it remains stable and predictable under a small scale load.

Separate chaining or bucket implementation offer multiple advantages of simplicity, stability under low/medium load and relatively easy updates and deletes. However, the approach may suffer from minor pointer overhead and additional space occupation.

On the other hand, open-addressing may look cache-friendly due to its array-only structure, but deletion and clustering problem makes operations trickier. Besides, the performance degrades as load factor approaches 1.

In this implementation, separate chaining method is used, since the number of cities (the keys) are mostly a few hundreds, but avoids clustering and tricky operations of insertion and deletion that poses a slight performance overhead.

Empirical collisions

With `mod` = 4099 on the Sweden dataset (52 cities, 75 connections) in the file trains.csv, a load factor of ≈ 0.013 and *two* non-empty buckets with chain length 2 are observed; all other buckets had length 1 or 0. Collisions are therefore rare and shallow, confirming that the unsigned-byte polynomial hash spreads city names well at this scale.

Graph generated from csv

The map is stored in CSV with lines `src, dst, travel time (minutes)`. Each row is bidirectional, where going in both directions is possible and takes same amount of travel time. When the program starts, the map is loaded and parsed. For each row, city nodes are created if they do not already exist, and the two cities are then connected bidirectionally by creating two edges: one from `src` to `dst` and another from `dst` to `src`. The code snippet is seen as below.

```
// Bidirectional connection for the undirected map
void connect(city *src, city *dst, int dist) {
    connect_directed(src, dst, dist);  // src -> dst
    connect_directed(dst, src, dist);  // dst -> src
}
```

Depth First Search

Similarly to trees, searching the optimal path through a graph between two nodes can be performed using depth-first search. Instead of starting from root node in trees, the algorithm starts from the source node (in this case source city) and advance forward by using depth-first traversal toward the destination node (destination city). The traversal algorithm enumerates paths recursively by following each connection from the adjacency list from the current city. Each weights are treated as non-negative travel minutes and the length in minute of the shortest path.

DFS with time limit (DFS-LIMIT)

Starting from source city, every possible connection to the destination city is checked, where the path with minimum travel time is returned. However,

since the graph is bidirectional and contains cycles, the algorithm can oscillate between several adjacent nodes, resulting in an infinite loop. A simple solution is to introduce a time limit (variable `left`). Each time an edge is traversed, the time cost is deducted from this limit. When time limit runs out, the current path is discarded, and the algorithm continues checking other paths, avoiding infinite loops. However, this approach does not detect cycles and will not prevent the algorithm to revisit same nodes, as long as the time limit allows.

Code implementation

In terms of code implementation, the algorithm first checks if the destination city is reached (`from == to`) which will return 0. Otherwise, its directed adjacent cities are iterated and will only recursively traverse if time limit is greater than the time cost to travel to the adjacent node (`nxt->dist` $\leq$ `left`). After recursive traversals from all possible adjacent cities to destination, all possible shortest time costs are compared, whereas the smallest one is returned. If no possible shortest time costs exists, the default value -1 is returned, which means that no adjacent city yielded a feasible path.

```
static int dfs_shortest_limit(city *from, city *to, int left) {
    if (from == to) return 0;  // Reached destination
    int sofar = -1;  // Best distance found (-1 = none)
    // Try all adjacent cities
    for (conn *nxt = from->adj; nxt; nxt = nxt->next) {
        if (nxt->dist <= left) {  // Only if within limit
            // Recurse with reduced budget
            int d = dfs_shortest_limit(nxt->dst, to, left - nxt->dist);
            if (d >= 0) {  // Valid path found
                int cand = d + nxt->dist;  // Total distance
                if (sofar < 0 || cand < sofar) sofar = cand;  // Update best
            }
        }
    }
    return sofar;  // Return best distance or -1
}
```

Benchmark results and analysis

Setup. For benchmarking the DSF traversal algorithm, a timeout limit of 600 minutes is set for the process. The algorithm is executed on nine representative source-destination pairs on a map with Swedish railroad network, where the shortest distances in minutes and execution times (ms) are recorded if available.

Observed results (selected). The results observed are as following:

- Malmö → Göteborg: **153** minutes ⇒ **0.38 ms**
- Göteborg → Stockholm: **211** minutes ⇒ **372.49 ms**
- Stockholm → Sundsvall: **327** minutes ⇒ **248.18 ms**
- Sundsvall → Umeå: **190** minutes ⇒ **0.00 ms**

Other queries timed out with the given budgets:

- Malmö → Stockholm (budget 700): **TIMEOUT**
- Stockholm → Umeå (budget 1200): **TIMEOUT**
- Göteborg → Sundsvall (budget 1000): **TIMEOUT**
- Umeå → Göteborg (budget 1200): **TIMEOUT**
- Göteborg → Umeå (budget 1200): **TIMEOUT**

Why timeouts occur (not infinite loops, but explosion). The reason to timeouts is that the travel time of the optimal path between two cities is larger than the given time limit. In this case, since it doesn't exist a path with shorter travel time than the optimal one, the algorithm fails to find any valid path by traversing within the time limit.

Since this implementation lacks visited or cyclic path detection, this algorithm enumerates repeated prefixes and detours, which will eventually stop when the remaining limit runs out, but the number of explored paths before is in most time unnecessarily and intolerably large. The time complexity can be described as:

$$\text{paths explored} \approx \mathcal{O}(b^{\lfloor \text{time_limit}/\text{min_edge} \rfloor}),$$

where b is the average branching factor, `time_limit` is the time limit for traversal and `min_edge` is minimal amount of edges. Practically, this results into a large loop until external timeout is reached.

Timeout limit sizing trade-offs. Because of the large time complexity order, it is therefore crucial to choose the optimal timeout for the algorithm. If the timeout is set too short—shorter than the unknown optimal path—the algorithm may terminate too early, yielding no feasible solution. However, if the limit is set too large, the search space grows exponentially where detours are permitted, resulting in a long runtime, often in an intolerable and impractical execution time.

Despite the simplicity of DSF traversal with time limit, lack of cycle/visited control result in an exponential growth with a growing number of cities.

DFS with Loop-detection (DFS-LOOP)

Another traversal implementation uses a cycle-detection mechanism by maintaining a set of visited cities, using a `path[]` array. Instead of checking the remaining timeout limit before traversing to an adjacent city, the adjacent city is checked if it's included in the set. The city will only be traversed if it doesn't include in the set. This implementation uses cycle-detection mechanism to avoid infinite loop oscillation and therefore dramatically reduces amount of redundant cycle traversal. However, the membership test is linear $\mathcal{O}(k)$ per edge in current depth and the algorithm still does not remember solutions to subproblems reached through different prefixes.

Code implementation

In code implementation, the algorithm begins by checking if destination is reached, which is similar to the previous one. However, when trying all adjacent cities one by one, instead of checking timeout, the adjacent city is checked if it exists in the visited array using a linear array scan in function `in_path`. This procedure has a time complexity of $\mathcal{O}(k)$ in a visited array of k elements. If the city is not visited, it will be visited by recursively searching the shortest path from the given city, as well as pushing it into the visited array. In this case, cities that already have been traversed will not be re-visited, guaranteeing termination without timeout limit. In the end, a minimum distance is calculated from each city to destination, which is thereafter returned. Once the recursion returns, the most recently added city is no longer considered part of the current path and can be revisited through a different path.

```c
static int in_path(city **stk, int k, city *x) {
    for (int i = 0; i < k; ++i) {
        if (stk[i] == x) return 1;  // Found in path - would create cycle
    }
    return 0;  // Not in path - safe to explore
}

static int dfs_shortest_loop(city *from, city *to, city **path, int k) {
    if (from == to) return 0;  // Reached destination
    int sofar = -1;  // Best distance found

    // Try all adjacent cities
    for (conn *nxt = from->adj; nxt; nxt = nxt->next) {
        if (!in_path(path, k, nxt->dst)) {  // Only if not in current path
            path[k] = nxt->dst;  // Add to path
            int d = dfs_shortest_loop(nxt->dst, to, path, k+1);  // Recurse
            if (d >= 0) {
```

```
                int cand = d + nxt->dist;
                if (sofar < 0 || cand < sofar) sofar = cand;
            }
            // Path[k] automatically "removed" when function returns
        }
    }
    return sofar;
}
```

Benchmark results and analysis

Setup. DFS algorithm with loop detection is executed on nine representative routes in the Swedish railroad network map, while the shortest distance (in minutes) and execution time (in ms) are recorded.

Results. The execution results are shown as below. Since the algorithm relies on cycle detection, timeout problem does therefore not occur.

- Malmö → Göteborg: **153** minutes ⇒ **150.52 ms**
- Göteborg → Stockholm: **211** minutes ⇒ **71.03 ms**
- Malmö → Stockholm: **273** minutes ⇒ **149.11 ms**
- Stockholm → Sundsvall: **327** minutes ⇒ **105.70 ms**
- Stockholm → Umeå: **517** minutes ⇒ **149.19 ms**
- Göteborg → Sundsvall: **515** minutes ⇒ **126.37 ms**
- Sundsvall → Umeå: **190** minutes ⇒ **375.30 ms**
- Umeå → Göteborg: **705** minutes ⇒ **128.52 ms**
- Göteborg → Umeå: **705** minutes ⇒ **182.56 ms**

Analysis. The loop-detection algorithm guarantees termination without a timeout limit and eliminates infinite oscillations on bidirectional edges, but it still explores all possible paths between the endpoints before an optimal one is found. As local branching increase, for example around major hubs, the number of simple paths grows rapidly, which causes longer distance between cities into longer execution times. The asymmetric times for $A \to B$ and $B \to A$ arise from adjacency order and branching structure encountered during DFS. If the graph is undirected, but the search order and available detours along each direction differ changes how much pruning occurs in practice. In general, loop-avoidance dramatically improves performance over timeout-limit DFS, but remains sensitive to graph density and hub connectivity because it does not remember subproblem results across different prefixes.

Benchmark comprison (DFS-LIMIT vs DFS-LOOP)

Benchmark results across two different algorithm approaches, DFS with timeout limit and DSF with loop detection, shows that DFS with loop detection outperforms timeout limit approach between cities with multiple connection traversal in optimal paths, since cyclic paths are pruned out, narrowing the number of traversal combinations. However, on nearby linear queries, timeout approach outperforms loop detection one because pruning by timeout limit stops exploration quickly, while loop detection algorithm adds overhead of traversing multiple adjacent nodes before conforming the optimal path. The execution time and timeout can be shown in Table 1.

Additionally, DFS using loop detection consistently finishes on all tested routes, while timeout approach times out on five of the given nine routes where the limit is set too low. Removing cycles guarantees termination, but it does not always guarantee minimal execution time.

Asymmetric times between $A \rightarrow B$ and $B \rightarrow A$ persist: DFS proceeds in adjacency order and encounters different branching patterns per direction, which changes how soon good candidates are found and how much search is pruned in practice.

The results of the both algorithms can be visualized as below:

Route	**DFS-LIMIT (ms)**	**DFS-LOOP (ms)**
Malmö → Göteborg	**0.38**	150.52
Göteborg → Stockholm	372.49	**71.03**
Malmö → Stockholm	`TIMEOUT`	149.11
Stockholm → Sundsvall	248.18	**105.70**
Stockholm → Umeå	`TIMEOUT`	149.19
Göteborg → Sundsvall	`TIMEOUT`	126.37
Sundsvall → Umeå	**0.00**	375.30
Umeå → Göteborg	`TIMEOUT`	128.52
Göteborg → Umeå	`TIMEOUT`	182.56

Table 1: Comparison of DFS-LIMIT and DFS-LOOP on selected routes

Improvement: DFS with visited flag (DFS-VISITED)

The two previous approaches can be improved by adding a visited flag on city structure, instead of keeping an array with visited cities, as well as keeping the timeout limit as a guardrail. Visiting a city marks it visited by switching the flag to `true`. When leaving the city and exploring other paths, the visited flag is unmarked. Visiting a city checks the flag and it will only be visited if the flag shows unvisited. Additionally, the timeout limit prune out unproductive detours, despite being cycle-less.

Because of this, backtracking becomes possible for reconstructing the shortest path, since a city is unmarked after exploring its ongoing connections so other branches that does not pass this city on their prefix can consider it later.

This improves overall performance by reducing the linear search of the visited array in $\mathcal{O}(k)$ with constant time $\mathcal{O}(1)$ flag check for cycle detection. Using this approach, bouncing across already explored segments before timeout is reached can be avoided with a computational cheap cycle detection.

However the time complexity remains exponential in depth $O(b^d)$, where b is average branching and d is average depth, because it may still enumerate many simple paths in dense connection region.

Code implementation

The code implementation is almost similar to DFS using timeout approach, but the difference lies in the visited flag detection logic. When the current city is visited, its visited flag is marked as true, before recursively visit its adjacent cities. Afterwards, when leaving the node, the visited flag is unmarked, which makes it possible for paths with different prefixes to visit it.

```
static int dfs_shortest_visited(city *from, city *to, int left) {
    if (from == to) return 0;       // Base case: reached destination
    if (left < 0) return -1;        // Budget exceeded
    if (from->visited) return -1;   // Cycle on current recursion stack

    from->visited = true;           // Mark: enter node
    int sofar = -1;                 // Best distance found so far

    for (conn *nxt = from->adj; nxt; nxt = nxt->next) {
        if (nxt->dist <= left) {    // Budget pruning
            int d = dfs_shortest_visited(nxt->dst, to, left - nxt->dist);
            if (d >= 0) {
                int cand = d + nxt->dist;
                if (sofar < 0 || cand < sofar) sofar = cand;  // Keep best
            }
        }
    }
    from->visited = false;          // Unmark: leave node (backtrack)
    return sofar;
}
```

Benchmark results and analysis

Results. The results show that all tested routes complete quickly in millisecond time range, with larger values (e.g. Stockholm to Umeå at 7.92 ms and Göteborg to Umeå at 9.92 ms) reflecting longer paths and more branching near hubs.

- Malmö → Göteborg: **153** minutes ⇒ **0.01 ms**
- Göteborg → Stockholm: **211** minutes ⇒ **0.07 ms**
- Malmö → Stockholm: **273** minutes ⇒ **0.40 ms**
- Stockholm → Sundsvall: **327** minutes ⇒ **0.10 ms**
- Stockholm → Umeå: **517** minutes ⇒ **7.92 ms**
- Göteborg → Sundsvall: **515** minutes ⇒ **3.87 ms**
- Sundsvall → Umeå: **190** minutes ⇒ **0.00 ms**
- Umeå → Göteborg: **705** minutes ⇒ **1.33 ms**
- Göteborg → Umeå: **705** minutes ⇒ **9.92 ms**

Analysis. Adding a *visited* flag to the budget-limited DFS removes back-and-forth oscillations on bidirectional edges and avoids a large portion of redundant exploration. The timeout limit only serves as a guardrail against excessively deep detours, while correctness and termination no longer rely on the timeout limit alone. Instead, the *mark–recurse–unmark* pattern ensures the search enumerates only cycle-less, which is dramatically more efficient in practice for this map.

Better solution: DFS with branch-and-bound (DFS-BB)

Another solution extends loop detection DFS approach with a dynamic upper bound (variable `best_so_far`) that is carried down the recursion and tightened whenever a better complete path is discovered. Any connection whose cost already exceeds the remaining bound is pruned immediately where entire sub-trees are skipped if they have larger cost than the currently best one. For instance, if the best path found so far is 500 minutes and another edge costs 600 minutes, the edge will be skipped entirely, since it cannot possibly yield a smaller time cost value.

This approach still prevents cycles by refusing to visit the already visited nodes in the path.

Code implementation

The code implementation for branch-bound search is similar to DFS with loop detection algorithm, but contains an additional pruning mechanism. The algorithm maintains two variables: `sofar`, which represents the length of the best complete path found so far from the source to the destination, and `found` a flag indicating whether any valid path to the destination has been discovered.

When trying all adjacent cities, cities that have already been traversed in path are avoided, which is similar to DFS with loop detection algorithm. Additionally, a connection is skipped if its distance alone is greater than or equal to `sofar`, since any path through that connection cannot produce shorter distance. The remaining adjacent cities are traversed by calling the algorithm recursively by calculating the best equivalent distance from each adjacent city to the destination `sofar - nxt->dist` and passing it to `best_so_far` parameter. If a shortest valid path is calculated, best distance is updated to the smallest one, as the flag `found` is set.

Setting the parameter `best_so_far` to -1 disables the algorithm to initially prune by shortest given distance, and is useful in initial conditions, as the shortest distance remains unknown.

```
static int dfs_bb_inner(city *from, city *to, city **path, int k, int best_so_far) {
    if (from == to) return 0;  // Reached destination

    int sofar = best_so_far;  // Current best bound
    int found = 0;  // Track if we found any valid path

    // Try all adjacent cities
    for (conn *nxt = from->adj; nxt; nxt = nxt->next) {
        // Skip if: (1) already in path (cycle) OR (2) edge alone exceeds bound
        if (in_path(path, k, nxt->dst)) continue;  // Avoid cycles
        if (sofar >= 0 && nxt->dist >= sofar) continue;  // pruning!

        path[k] = nxt->dst;  // Add to path
        // Recurse with updated bound (remaining budget for subtree)
        int d = dfs_bb_inner(nxt->dst, to, path, k+1,
                             sofar < 0 ? -1 : (sofar - nxt->dist));
        if (d >= 0) {  // Valid path found
            int cand = d + nxt->dist;
            if (sofar < 0 || cand < sofar) {
                sofar = cand;  // Update bound
                found = 1;
            }
        }
    }
```

```
    return found ? sofar : -1;  // Return best or -1 if no path
}

// Wrapper to start branch-and-bound search
static int dfs_shortest_bb(city *from, city *to, city **path, int k) {
    return dfs_bb_inner(from, to, path, k, -1);  // Start with no bound
}
```

Benchmark results and analysis

Results.

- Malmö → Göteborg: **153** minutes ⇒ **0.01 ms**
- Göteborg → Stockholm: **211** minutes ⇒ **0.02 ms**
- Malmö → Stockholm: **273** minutes ⇒ **0.04 ms**
- Stockholm → Sundsvall: **327** minutes ⇒ **0.06 ms**
- Stockholm → Umeå: **517** minutes ⇒ **0.57 ms**
- Göteborg → Sundsvall: **515** minutes ⇒ **0.67 ms**
- Sundsvall → Umeå: **190** minutes ⇒ **0.00 ms**
- Umeå → Göteborg: **705** minutes ⇒ **0.10 ms**
- Göteborg → Umeå: **705** minutes ⇒ **3.72 ms**

Analysis. The benchmark results shows that the branch and bound approach consistently achieves low execution time. For long routes, the execution time is roughly 0.01 to 4 ms. Compared to DFS-loop algorithm, the execution time is 50 to 100 times less. The reason is that a dynamic upper bound `best-so-far` is propagated down the recursion and any edge whose travel distance already exceeds the remaining bound is pruned out. Once an initial complete path is found that is less than the bound distance, the bound is tightened and eliminates any other possible larger paths. The details could be seen in table 2.

However, the time complexity is still exponential in the worst case $O(b^d)$, with an average number b of branches and d in depth. The reason is that this algorithm still enumerates all paths in the worst case, if the branches are equally long. From a practical perspective, the complexity coefficient grows slower compared to without the approach, since the bound becomes informative early by focusing the search on reasonably optimal branches instead of trying detours.

Route	DFS-LIMIT	DFS-LOOP	DFS-VISITED	DFS-BB
Malmö → Göteborg	0.38	150.52	**0.01**	**0.01**
Göteborg → Stockholm	372.49	71.03	0.07	**0.02**
Malmö → Stockholm	`TIMEOUT`	149.11	0.40	**0.04**
Stockholm → Sundsvall	248.18	105.70	0.10	**0.06**
Stockholm → Umeå	`TIMEOUT`	149.19	7.92	**0.57**
Göteborg → Sundsvall	`TIMEOUT`	126.37	3.87	**0.67**
Sundsvall → Umeå	**0.00**	375.30	**0.00**	**0.00**
Umeå → Göteborg	`TIMEOUT`	128.52	1.33	**0.10**
Göteborg → Umeå	`TIMEOUT`	182.56	9.92	**3.72**

Table 2: Runtime comparison across four DFS variants (milliseconds). Bold = fastest per row. DFS-LIMIT stands for DFS using timeout limit, DFS-LOOP for DFS using loop detection, DFS-VISITED for DFS with visited flag and DFS-BB for DFS with branch-and-bound

Things to ponder

However, even the DFS with branch and bound mechanism can produce long execution time in a larger map. By measuring execution times for route searching beginning from Malmö to cities farther away on the Sweden map (with approximately 52 cities), the result shows an exponential growth in execution time. Scaling into a larger Europe map (approximately 500 cities) without altering the route search query also increases the execution time significantly. Finally, memoization method for previously solved sub-problems becomes important in terms of improving efficiency, as it avoids redundant re-computations and reduces the overall search cost.

Malmö → cities at increasing distances (Sweden, 52 nodes)

Using the currently fastest DFS algorithm (branch-and-bound) applied to a growing number of hop counts (number of traveled connection), executed measurement time form a exponential growth pattern, which is shown in Table 3.

Judging the measurement data from Table 3, execution time rises sharply with hop count. Within 9 hops, the execution time is sub-milliseconds, but rises significantly to tens of milliseconds as the hops reaches over 13 hops. sub-0.05 ms for 3–9 hops, then tens of ms for 13–16 hops. This reflects the exponential path unwrapping - despite optimizations with branch-and-bound pruning, since the search still enumerates many simple paths on long routes through dense, bidirectional regions.

Category (hops)	Destination	Distance (min)	Time (ms)
Short (1–2)	Lund	13	0.01
	Helsingborg	48	0.01
	Hässleholm	43	0.01
Medium (3–9)	Göteborg	153	0.03
	Jönköping	148	0.02
	Stockholm	273	0.04
Long (10–11)	Uppsala	324	0.07
	Gävle	383	0.15
Long (12)	Sundsvall	600	2.84
Very long (13)	Östersund	612	13.35
	Umeå	790	13.73
Extreme (14+)	Boden	976	46.67
	Luleå	1002	50.04
	Gällivare	1095	67.35
	Kiruna	1162	83.50

Table 3: DFS-BB runtime from Malmö to cities at increasing distances (Sweden map)

Increasing the map while fixing search query

Increasing the number of nodes also increases the number of branching, where each path expansion consider more adjacent cities, which increases the number of candidate paths to compare. Besides, long-distance trips traverse more intermediate cities, which increases the hop count d.

Because DFS explores paths rather than remembering cities as states, its effective work grows roughly in $\alpha\, b^{\,d}$ (α coefficient, branching factor b, depth d).

DFS implementation on an Europe-wide map

On a Europe-wide graph (500 cities) both b and d are higher, so the same DFS-BB method becomes impractical for long routes. For instance, Malmö→Kiruna on a larger European map required about 1.1×10^5 ms, $\mathcal{O}(10^5)$ ms, compared with a Sweden map in 80 ms (less than one second), consistent with the exponential sensitivity to d and with the much larger frontier of alternatives. While the Sweden map is manageable, a continent-scale map pushes the DFS approach beyond reasonable run times for long-distance queries.

Why memoization matters

The four different depth-first search algorithms, whether baseline, loop-detection, or branch-and-bound does not memoize and remember the solved subproblems. For instance, searching the optimal route between Malmö to Kiruna, the algorithm often reaches Stockholm, but the frequent search route between Stockholm to Kiruna (889 minutes) is not remembered and therefore recomputed multiple times. Before exploring alternatives from Norrköping (e.g. via Katrineholm) prior to Stockholm, remembering that the optimal route and its travel time between Stockholm to Kiruna would be a feasible optimization to cut down unnecessary recalculation and avoid repeated path explosion.

Conclusion

This report contains building and progression in optimal route searching in weighted bidirectional graphs from CSV data using a hash table with UTF-8 char array keys for $O(1)$ average name lookups, an adjacency-list city graph, and four different DFS traversal algorithm: DFS with timeout limit, DFS with loop-detection mechanism, DFS with visited flag and DFS with branch-and-bound mechanism.

Executing the algorithm in Sweden map (52 cities) shows that DFS with timeout limit is simple but fragile. Without the cycle detection logic, the timeout limit may either prune away valid path routes if it is set to small, or traverses cyclical if the budget is too large, resulting in long execution time.

An improvement to this algorithm is loop-detection DFS that guarantees termination and removes oscillations between visited cities, but it still enumerates multiple paths which makes runtime to rise significantly around dense hubs and longer routes. Adding a visited flag onto each city structure instead of keeping traversed cities as an array with a linear search optimizes cycle detection, but the traversal complexity remains unaffected.

The most effective of the given DFS algorithm is branch-and-bound DFS that reduces the search space by pruning out the connections that are longer than the shortest found path, using a dynamic best-so-far bound.

However, despite these improvements and optimizations, all DFS algorithms remain path-oriented and memoryless. As a result, the solves subproblems are not remembered and memoized, so the same subproblem is recomputed over whenever the search reaches that city through a new path. This explains the exponential sensitivity to path depth and branching, resulting in an exponential time complexity $O(b^d)$ in worst case with b average branches each city and d depths. These algorithm quickly become impractical in continent-scale instances, where the execution time rises sharply and become intolerable long.